

Stat331 Assignment 1

Milagro Chen (21066922)

2026-06-06

Set up

Here are the libraries used:

```
library(data.table)
```

```
##
## Attaching package: 'data.table'

## The following object is masked from 'package:base':
##
##      %notin%
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.3      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::between()      masks data.table::between()
## x dplyr::filter()       masks stats::filter()
## x dplyr::first()        masks data.table::first()
## x lubridate::hour()     masks data.table::hour()
## x lubridate::isoweek()  masks data.table::isoweek()
## x lubridate::isoyear()  masks data.table::isoyear()
## x dplyr::lag()          masks stats::lag()
## x dplyr::last()         masks data.table::last()
## x lubridate::mday()     masks data.table::mday()
## x lubridate::minute()   masks data.table::minute()
## x lubridate::month()    masks data.table::month()
## x lubridate::quarter()  masks data.table::quarter()
## x lubridate::second()   masks data.table::second()
## x purrr::transpose()    masks data.table::transpose()
## x lubridate::wday()     masks data.table::wday()
## x lubridate::week()     masks data.table::week()
## x lubridate::yday()     masks data.table::yday()
## x lubridate::year()     masks data.table::year()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Question 1

First we clean the original csv file to be in accordance with the assignment. We want BB and TSXC to be percent change, and GVTB10 to be in monthly rates instead of yearly. We also drop the old columns and OTEX from the dataframe. On top of that, we want the very first row to be removed, as it has an undefined percentage change from the month previous(it doesn't have a previous month).

```
CAPM_S26 <- read.csv('~/.Documents/GitHub/Big-Ahh-R-Repo/Stat331/Stat331 A1/data/raw/CAPM_S26.csv')
capm_s26 <- CAPM_S26
capm_s26$bb_Percent <- (CAPM_S26$BB - lag(CAPM_S26$BB, 1) ) / lag(CAPM_S26$BB, 1)
capm_s26$tsxc_Percent <- (CAPM_S26$TSXC - lag(CAPM_S26$TSXC, 1) ) / lag(CAPM_S26$TSXC, 1)
capm_s26$gvtb10_monthly <- (((CAPM_S26$GVTB10 / 100) + 1) ** (1/12)) - 1

capm_s26 <- capm_s26[-1,]
```

Moving forward, we will define x and y to be the excess return of the TSXC and BB respectively:

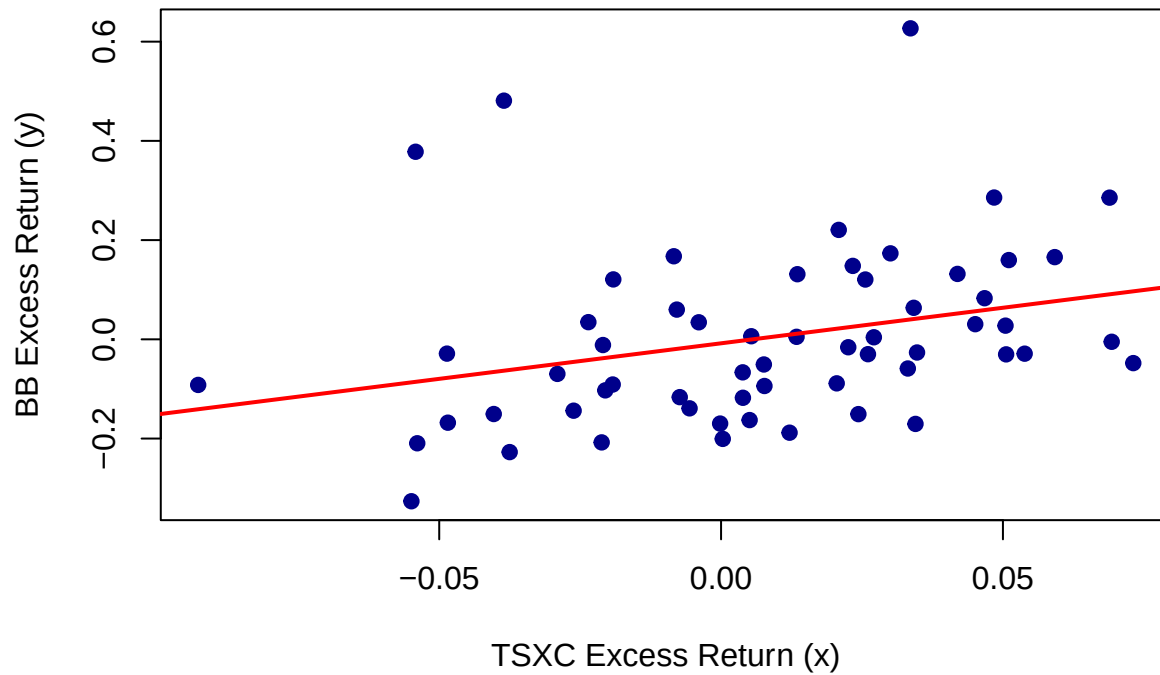
```
y <- capm_s26$bb_Percent - capm_s26$gvtb10_monthly
x <- capm_s26$tsxc_Percent - capm_s26$gvtb10_monthly
```

a)

Here is the plot. Fitting a line of best fit, we see the slope is positive, hinting that BB's beta is slightly positive. We will proceed to verify that claim, and see just how strong such a claim is. A linear model is appropriate, as the data roughly follows a linear pattern. There aren't too many outliers, but some months BB's excess returns are rather higher than TSXC.

```
plot(x, y,
     main="BB Excess Returns vs. Market Excess Returns",
     xlab="TSXC Excess Return (x)",
     ylab="BB Excess Return (y)",
     pch=19, col="darkblue")
abline(lm(y ~ x), col="red", lwd=2)
```

BB Excess Returns vs. Market Excess Returns



b)

Using R, we find the correlation coefficient to be 0.2959326:

```
cor(x,y)
```

```
## [1] 0.2959326
```

c)

Manually checking, we arrive at the same number, 0.2959326:

```
xbar <- mean(x)
```

```
ybar <- mean(y)
```

```
xvar <- sum((x - xbar)**2)
```

```
yvar <- sum((y - ybar)**2)
```

```
rho <- (sum((x - xbar)*(y - ybar)))/(sqrt(yvar*xvar))
```

```
rho
```

```
## [1] 0.2959326
```

d)

Performing the linear regression we get this summary:

```
slr <- lm(y ~ x)
```

```
summary(slr)
```

```
##
```

```
## Call:
```

```
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.23962 -0.09829 -0.04028  0.07948  0.58667
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.008004   0.022605  -0.354   0.7246
## x            1.432575   0.612471   2.339   0.0229 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1701 on 57 degrees of freedom
## Multiple R-squared:  0.08758,    Adjusted R-squared:  0.07157
## F-statistic: 5.471 on 1 and 57 DF,  p-value: 0.02286
```

e)

searching on <https://www.investing.com/equities/research-in-motion-ltd?cid=1055239>, we find that our model outputted 1.43 while the website has 1.47, and on a deeper dive it has beta = 1.55. This is explained by different methodology, such as slightly different time frame of data(daily,yearly,etc.), slight discrepancies in data(different sources), and sheer amount of data(we used 59 months, whereas the website could be using the full 60 months)

f)

Using R, we have this as our 95% confidence interval:

```
confint(slr)

##              2.5 %    97.5 %
## (Intercept) -0.05326971 0.0372623
## x            0.20612328 2.6590266
```

Specifically looking at the x variate, we see the interval is [0.20612328, 2.6590266], showing that the true beta is, with 95% probability, in between 0.20612328 and 2.6590266. Because the 95% confidence interval contains 1, we do not have sufficient statistical evidence to conclude that BB's stock volatility is significantly different from the market volatility

Manually, we get this same result:

```
t57_0975 <- qt(0.975, 57)
low <- summary(slr)$coefficients[2,1] - summary(slr)$coefficients[2,2] * t57_0975
high <- summary(slr)$coefficients[2,1] + summary(slr)$coefficients[2,2] * t57_0975
c(low,high)

## [1] 0.2061233 2.6590266
```

g)

Viewing our summary again:

```
summary(slr)

##
## Call:
## lm(formula = y ~ x)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.23962 -0.09829 -0.04028  0.07948  0.58667
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.008004   0.022605  -0.354   0.7246
## x              1.432575   0.612471   2.339   0.0229 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1701 on 57 degrees of freedom
## Multiple R-squared:  0.08758,    Adjusted R-squared:  0.07157
## F-statistic: 5.471 on 1 and 57 DF,  p-value: 0.02286
```

We see that the intercept, which is interpreted as the expected excess return of BB given that TSXC returns 0% after accounting for the risk free rate in a given month, AKA BB's alpha, is -0.008004, with a p-value of 0.7246, signalling that we failed to reject the null hypothesis of BB's alpha being 0. We can't conclude that BB has any alpha, statistically speaking.

h)

First let's show betahat and sigmahat:

```
betahat <- sum((x - xbar)*(y - ybar))/(xvar)
betahat
```

```
## [1] 1.432575
```

```
sigmahat <- sqrt(sum((residuals(slr))^2)/(length(x)-2))
sigmahat
```

```
## [1] 0.1700539
```

Using these the values above, let's show alphahat and SEbetahat:

```
alphahat <- ybar - xbar* sum((x - xbar)*(y - ybar))/(xvar)
alphahat
```

```
## [1] -0.008003707
```

```
SEbetahat <- sigmahat/sqrt(xvar)
SEbetahat
```

```
## [1] 0.6124708
```

Finally, let's manually test with hypothesis of beta = 0:

```
t_stat <- betahat / SEbetahat
p_value_beta <- 2 * (1 - pt(abs(t_stat), df = length(x) - 2))
p_value_beta
```

```
## [1] 0.02286395
```

Question 2

First we set up the data, with the true parameters given by the assignment, and setting up our matrix with code given by the assignment:

```

truebeta <- c(-200000, 45, -300, 10000, 150000)
truesigma <- 12000
A1_variates <- read.table('/~/Documents/GitHub/Big-Ahh-R-Repo/Stat331/Stat331 A1/data/raw/A1_variates (1)
                        stringsAsFactors = FALSE)
n <- nrow(A1_variates)
n

## [1] 24
X <- as.matrix(cbind(rep(1,n),A1_variates))

```

a)

Since we've set the random seed, we can generate a replicable random dataframe for the assignment:

```

set.seed(21066922)
simulatedresponse <- (as.vector(X %*% truebeta) + rnorm(n,0,truesigma))
simulatedresponse

## [1] 166281.206 153730.937 43338.698 193046.302 137185.704 10542.334
## [7] 14183.044 81275.444 115711.040 98465.901 39357.221 11103.992
## [13] 7704.852 82777.187 110100.768 88648.310 47059.937 161115.200
## [19] 160731.747 142684.940 123764.878 126090.364 173399.992 -5527.173

```

b)

Labelling our variables a little more, we run the multilinear model:

```

y <- simulatedresponse
size <- A1_variates$size
age <- A1_variates$age
employees <- A1_variates$employees
col <- A1_variates$col
mlr <- lm(y~size+age+employees+col)
summary(mlr)

##
## Call:
## lm(formula = y ~ size + age + employees + col)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22388  -5899   1651   7308  17474
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -161990.95   58235.05  -2.782  0.011889 *
## size          66.05      17.06    3.872  0.001027 **
## age         -447.16     392.25   -1.140  0.268453
## employees    7100.09    1657.67    4.283  0.000401 ***
## col         113817.43   54698.58    2.081  0.051221 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11500 on 19 degrees of freedom
## Multiple R-squared:  0.9709, Adjusted R-squared:  0.9648

```

```
## F-statistic: 158.5 on 4 and 19 DF,  p-value: 2.6e-14
```

c)

We now manually calculate it by definition and confirm that the model provides the same betashat:

```
betashat <- solve(t(X)%*%X)%*%t(X) %*% y
betashat
```

```
##                [,1]
## rep(1, n) -161990.95474
## size      66.05003
## age      -447.16037
## employees  7100.08632
## col      113817.42867
```

d)

Let's find the 95% confidence interval for the size variate:

```
confint(mlr)
```

```
##                2.5 %      97.5 %
## (Intercept) -283878.3251 -40103.5843
## size        30.3454    101.7547
## age        -1268.1392    373.8184
## employees   3630.5454   10569.6272
## col        -668.0088  228302.8661
```

Looking at the size row, we see the 95% confidence interval for size is [30.3454, 101.7547]. Since the true value of $\beta_1 = 45$ falls directly within our calculated 95% confidence interval of [30.3454, 101.7547], the interval is entirely consistent with the value used to generate the data.